



SQL y Pandas

SQL y Pandas: diferencias y similitudes

Actualmente, muchas organizaciones almacenan su información en archivos creados con la aplicación más utilizada mundialmente de manejo de hojas de cálculo: Microsoft Excel. Además, el análisis de datos es una rama que cada vez ha ido tomando más fuerza, y **Pandas**, una biblioteca de Python, se ha popularizado mucho debido a su simplicidad y eficiencia.

En esta lección revisaremos algunas similitudes y diferencias entre SQL y Pandas. Pero antes de iniciar con esta parte, veamos brevemente la forma en la que puedes cargar o recuperar información contenida en archivos de Excel, la cual nos servirá para los ejemplos que haremos en Google Colab, que seguramente ya has usado.

Para leer un archivo XLS, utilizamos la función `pd.read_excel`. Esta función recibe como parámetros el nombre del archivo XLS con todo y ruta, así como el nombre de la hoja. Recuerda que, en un archivo de Excel, podemos tener información clasificada en más de una hoja.

Puedes consultar detalles sobre esta función en la documentación oficial de Pandas. Encontrarás el enlace en la sección de Material adicional, al terminar esta lectura.

Supongamos que tienes un archivo de Excel con dos hojas de trabajo, una de **Empleados** y otra de **Autos**, solo por poner un ejemplo. A continuación, te mostramos las instrucciones para cargar los datos del archivo de Excel.

Lo primero que tenemos que hacer es importar Pandas y leer las hojas del archivo de Excel:

```
import pandas as pd
excel_empleados = pd.read_excel('records.xls', sheet_name='Empleados')
excel_autos = pd.read_excel('records.xls', sheet_name='Autos')
```

Y aquí se ve el resultado de la carga de archivos:

excel_empleados			
	EmplID	EmplName	EmpRole
0	1	John	CEO
1	2	Anna	Editor
2	3	Karen	Author

excel_autos					
	Marca	subMarca	Modelo	kilometraje	precio MXP
0	Honda	Civic	2015	67000	256000
1	Volkswagen	Jetta	2009	148500	110000
2	Toyota	Sienna	2011	140500	149900

Ahora que hemos visto cómo leer la información contenida en un archivo XLS, podemos proceder a revisar las analogías entre SQL y Pandas.

Consultas y analogías entre SQL y Pandas

Hablando del manejo de información, SQL es uno de los lenguajes más utilizados. Las operaciones básicas en una base de datos son **Crear** o insertar, **Leer** o recuperar, **Actualizar** y **Eliminar**, conocidas ampliamente como las operaciones **CRUD** por sus siglas en inglés (*Create, Read, Update y Delete*). Veamos a continuación cada una de ellas.

Insertar

La primera operación que revisaremos es la inserción de nueva información en nuestro *dataframe*. Primero veremos cómo esta operación es realizada en SQL, específicamente en MySQL, y posteriormente revisaremos cómo hacerlo en Pandas.

Consideremos que tienes la siguiente tabla en MySQL, donde `id` es un atributo de tipo entero, `auto_increment`, esto quiere decir que cada vez que insertes un registro nuevo, a éste se le asignará un valor consecutivo automáticamente:

```
CREATE TABLE Auto (  
  id INT NOT NULL PRIMARY KEY AUTO_INCREMENT,  
  Marca VARCHAR(20),  
  subMarca VARCHAR(20),  
  Modelo SMALLINT,  
  kilometraje INT,  
  precioMXP INT  
);
```

```
INSERT INTO Auto VALUES  
(NULL, "Honda", "Civic", 2015, 67000, 256000),  
(NULL, "VolksWagen", "Jetta", 2009, 148500, 110000),  
(NULL, "Toyota", "Sienna", 2011, 140500, 149900);
```

```
mysql> SELECT * FROM Auto;  
+----+-----+-----+-----+-----+-----+  
| id | Marca      | subMarca | Modelo | kilometraje | precioMXP |  
+----+-----+-----+-----+-----+-----+  
| 1  | Honda      | Civic    | 2015   | 67000       | 256000    |  
| 2  | VolksWagen | Jetta    | 2009   | 148500      | 110000    |  
| 3  | Toyota     | Sienna   | 2011   | 140500      | 149900    |  
+----+-----+-----+-----+-----+-----+  
3 rows in set (0.00 sec)
```

Si quieres insertar un nuevo registro, puedes hacerlo con este comando:

```
INSERT into Auto values (NULL, "Honda", "Fit", 2015, 110000,  
140000);
```

Con esto, la tabla quedaría de la siguiente forma:

```
mysql> SELECT * FROM Auto;
+----+-----+-----+-----+-----+-----+
| id | Marca      | subMarca | Modelo | kilometraje | precioMXP |
+----+-----+-----+-----+-----+-----+
| 1  | Honda      | Civic    | 2015   | 67000        | 256000    |
| 2  | Volkswagen | Jetta     | 2009   | 148500       | 110000    |
| 3  | Toyota     | Sienna    | 2011   | 140500       | 149900    |
| 4  | Honda      | Fit       | 2015   | 110000       | 140000    |
+----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Ahora, veamos cómo puedes hacerlo con Pandas.

Primero importamos la biblioteca Pandas y cargamos, desde nuestro archivo de Excel, la tabla de autos:

```
import pandas as pd
excel_autos = pd.read_excel('records.xls', sheet_name='Autos')
```

Ahora, creamos una nueva Serie con los valores que queremos agregar a la tabla:

```
new_row = pd.Series(["Honda", "Fit", 2015, 110000, 140000],
index = ["Marca", "subMarca", "Modelo", "kilometraje",
"precio MXP"])
```

En Pandas no podemos insertar una fila directamente a un *dataframe*, pero podemos **concatenar** dos *dataframes*, idealmente con el mismo "esquema". Por eso, a la Serie le asignamos los mismos índices que tiene el *dataframe* cargado desde Excel.

Ahora tenemos que convertir nuestra Serie en un *dataframe* para poder concatenarla posteriormente:

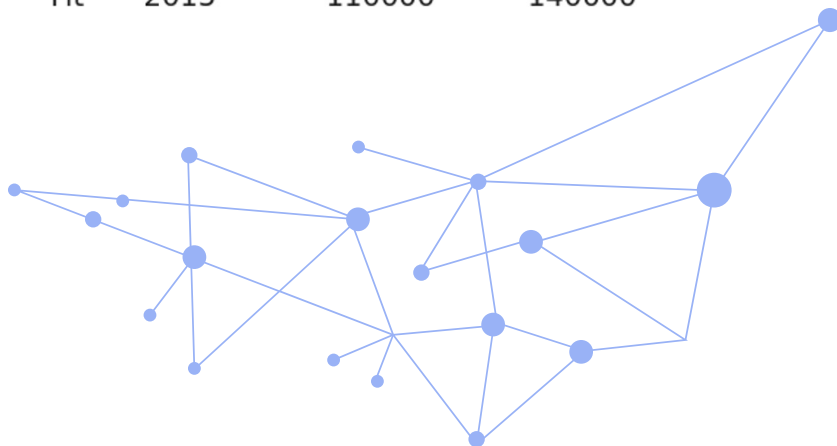
```
row_df = pd.DataFrame([new_row])
```

Aquí, `row_df` ya es un *dataframe* que tiene “el mismo esquema” que el *dataframe* de autos. Podemos hacer la concatenación usando la operación `pandas.concat`:

```
excel_autos = pd.concat([ excel_autos, row_df], ignore_index=True)
```

Aquí, el orden en que hagas el `concat` afecta cómo quedarán los índices. Si no usamos la opción `ignore_index=True`, podemos tener dos veces el mismo índice. Por eso usamos esa opción, para que los índices queden bien estructurados. Aquí puedes ver el resultado de la ejecución de todos estos comandos:

	Marca	subMarca	Modelo	kilometraje	precio MXP
0	Honda	Civic	2015	67000	256000
1	Volkswagen	Jetta	2009	148500	110000
2	Toyota	Sienna	2011	140500	149900
3	Honda	Fit	2015	110000	140000



Nota: Podrás observar que los índices en MySQL y en Pandas no coinciden. En MySQL, los índices inician en uno y no en cero. Esto se debe a que, en MySQL, definimos el campo id como un entero, no nulo, auto incrementado. Por defecto, no podemos tener un valor de 0 en esta columna. Se pueden hacer algunas modificaciones para lograr esto, pero no es recomendable.

Ahora revisemos cómo insertar más de una fila a la vez. Primero veamos cómo hacerlo en MySQL:

```
INSERT INTO Auto VALUES
(NULL, "Mazda", "CX-5", 2017, 65000, 265000),
(NULL, "Suzuki", "Vitara", 2016, 85000, 185000),
(NULL, "Audi", "A4", 2010, 185000, 225000);
```

Esto nos deja la tabla de la siguiente manera:

id	Marca	subMarca	Modelo	kilometraje	precioMXP
1	Honda	Civic	2015	67000	256000
2	Volkswagen	Jetta	2009	148500	110000
3	Toyota	Sienna	2011	140500	149900
4	Honda	Fit	2015	110000	140000
5	Mazda	CX-5	2017	65000	265000
6	Suzuki	Vitara	2016	85000	185000
7	Audi	A4	2010	185000	225000

Ahora veamos cómo hacerlo con Pandas.

Primero, recuerda que puedes concatenar un *dataframe* con otro *dataframe*. Por eso, necesitamos que los nuevos registros a insertar sean un *dataframe*. Lo que tenemos que hacer es crear una Serie nueva para cada registro nuevo, luego crear el *dataframe* concatenando las Series, y finalmente aplicar la transpuesta para que nos quede el *dataframe* en el formato del que ya teníamos.

```
new_row1 = pd.Series(["Mazda", "CX-5", 2017, 65000, 265000])
new_row2 = pd.Series(["Suzuki", "Vitara", 2016, 85000,
185000])
new_row3 = pd.Series(["Audi", "A4", 2010, 185000, 225000])
new_rows_df = pd.concat([new_row1,new_row2,new_row3],axis=1)
new_rows_df=new_rows_df.T
```

	0	1	2	3	4
0	Mazda	CX-5	2017	65000	265000
0	Susuki	Vitara	2016	85000	185000
0	Audi	A4	2010	185000	225000

Ahora, para que la concatenación de los *dataframes* quede como deseamos, tenemos que renombrar las columnas para que coincidan con las del *dataframe* de los autos previamente almacenados:

```
new_rows_df.columns = ["Marca", "subMarca", "Modelo",
"kilometraje", "precio MXP"]
```

Ahora nuestro *dataframe* ya está listo para concatenarlo con los datos previamente almacenados:

	Marca	subMarca	Modelo	kilometraje	precio MXP
0	Mazda	CX-5	2017	65000	265000
0	Susuki	Vitara	2016	85000	185000
0	Audi	A4	2010	185000	225000

Este *dataframe* tiene un pequeño detalle: **todos los índices están en cero**. Si queremos que quede consistente, podemos resetear los índices. Esto lo logramos con la función `reset_index(drop=True)`. Esta opción se usa para evitar que el índice actual se convierta en una columna nueva del *dataframe*.

Esta operación es opcional, dado que, al concatenar con los datos previos, podemos volver a reordenar los índices.

```
new_rows_df.reset_index(drop=True)
```

	Marca	subMarca	Modelo	kilometraje	precio MXP
0	Mazda	CX-5	2017	65000	265000
1	Susuki	Vitara	2016	85000	185000
2	Audi	A4	2010	185000	225000

Ahora podemos concatenar nuestro recién creado *dataframe* con el que teníamos previamente:

```
excel_autos = pd.concat([excel_autos,new_rows_df], ignore_index=True)
```

Finalmente, nuestro *dataframe* nos queda de la siguiente manera:

excel_autos

	Marca	subMarca	Modelo	kilometraje	precio MXP
0	Honda	Civic	2015	67000	256000
1	VolksWagen	Jetta	2009	148500	110000
2	Toyota	Sienna	2011	140500	149900
3	Honda	Fit	2015	110000	140000
4	Mazda	CX-5	2017	65000	265000
5	Susuki	Vitara	2016	85000	185000
6	Audi	A4	2010	185000	225000

■ Leer o recuperar

Siguiendo con nuestro ejemplo de los autos en MySQL, veremos cómo seleccionar los registros que cumplan con un cierto criterio, por ejemplo, los autos de una marca, como “Honda”.

Nuestra tabla tiene la siguiente información:

id	Marca	subMarca	Modelo	kilometraje	precioMXP
1	Honda	Civic	2015	67000	256000
2	Volkswagen	Jetta	2009	148500	110000
3	Toyota	Sienna	2011	140500	149900
4	Honda	Fit	2015	110000	140000
5	Mazda	CX-5	2017	65000	265000
6	Suzuki	Vitara	2016	85000	185000
7	Audi	A4	2010	185000	225000

La consulta para mostrar los autos Honda sería:

```
SELECT * FROM Auto WHERE Marca = "Honda";
```

Lo que produce el siguiente resultado:

```
mysql> SELECT * FROM Auto WHERE Marca = "Honda";
+----+-----+-----+-----+-----+-----+
| id | Marca | subMarca | Modelo | kilometraje | precioMXP |
+----+-----+-----+-----+-----+-----+
| 1  | Honda | Civic    | 2015   | 67000       | 256000    |
| 4  | Honda | Fit      | 2015   | 110000      | 140000    |
+----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

Si quisiéramos solo algunos atributos del resultado, lo podemos hacer con la siguiente consulta:

```
SELECT subMarca, Modelo FROM Auto WHERE Marca = "Honda";
```

Que da el siguiente resultado:

```
mysql> SELECT subMarca, Modelo FROM Auto WHERE Marca = "Honda";
+-----+-----+
| subMarca | Modelo |
+-----+-----+
| Civic    | 2015   |
| Fit      | 2015   |
+-----+-----+
2 rows in set (0.00 sec)
```

Si queremos solo algunos atributos de todos los autos, utilizamos:

```
SELECT subMarca, Modelo FROM Auto;
```

Lo que da como resultado:

```
mysql> select subMarca, Modelo from Auto;
+-----+-----+
| subMarca | Modelo |
+-----+-----+
| Civic    | 2015   |
| Jetta    | 2009   |
| Sienna   | 2011   |
| Fit      | 2015   |
| CX-5     | 2017   |
| Vitara   | 2016   |
| A4       | 2010   |
+-----+-----+
7 rows in set (0.00 sec)
```

Si quisiéramos, por ejemplo, los autos Honda con kilometraje menor a 100,000, la consulta sería:

```
SELECT * FROM Auto WHERE marca = "Honda" and kilometraje < 100000;
```

Lo que da como resultado:

```
mysql> SELECT * FROM Auto WHERE marca = "Honda" and kilometraje < 100000;
+----+-----+-----+-----+-----+-----+
| id | Marca | subMarca | Modelo | kilometraje | precioMXP |
+----+-----+-----+-----+-----+-----+
| 1  | Honda | Civic    | 2015   | 67000       | 256000    |
+----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Ahora hagamos todo esto con Pandas.

Recordemos el contenido actual de nuestro *dataframe*:

	Marca	subMarca	Modelo	kilometraje	precio MXP
0	Honda	Civic	2015	67000	256000
1	VolksWagen	Jetta	2009	148500	110000
2	Toyota	Sienna	2011	140500	149900
3	Honda	Fit	2015	110000	140000
4	Mazda	CX-5	2017	65000	265000
5	Susuki	Vitara	2016	85000	185000
6	Audi	A4	2010	185000	225000

Para seleccionar los autos de la marca Honda, ejecutamos la instrucción:

```
excel_autos.loc[excel_autos["Marca"]=="Honda"]
```

Lo que arroja el siguiente resultado:

	Marca	subMarca	Modelo	kilometraje	precio MXP
0	Honda	Civic	2015	67000	256000
3	Honda	Fit	2015	110000	140000

Como puedes observar, `excel_autos["Marca"]=="Honda"` regresa una serie de valores booleanos (`True` | `False`) y, con base en ese valor, se decide si una fila forma parte del resultado.

Para mostrar solo algunas columnas del último resultado, utilizamos la instrucción:

```
excel_autos.loc[excel_autos["Marca"]=="Honda"] .  
loc[:, ["subMarca", "Modelo"]]
```

Obteniendo el resultado:

	subMarca	Modelo
0	Civic	2015
3	Fit	2015

Si queremos ver solo algunos atributos de todos los autos, podemos usar:

```
excel_autos[["subMarca", "Modelo"]]
```

Lo que da como resultado:

	subMarca	Modelo
0	Civic	2015
1	Jetta	2009
2	Sienna	2011
3	Fit	2015
4	CX-5	2017
5	Vitara	2016
6	A4	2010

Y, finalmente, la consulta para ver autos Honda con menos de 100,000 kilómetros, la podemos escribir como:

```
excel_autos.loc[ (excel_autos["Marca"]=="Honda") &  
(excel_autos["kilometraje"] < 100000 ) ]
```

■ Actualizar

Otra de las operaciones básicas para una fuente de información son las actualizaciones. Sigamos revisando nuestro ejemplo en MySQL, para después ver cómo se hace en Pandas.

Partimos de la siguiente información:

id	Marca	subMarca	Modelo	kilometraje	precioMXP
1	Honda	Civic	2015	67000	256000
2	Volkswagen	Jetta	2009	148500	110000
3	Toyota	Sienna	2011	140500	149900
4	Honda	Fit	2015	110000	140000
5	Mazda	CX-5	2017	65000	265000
6	Suzuki	Vitara	2016	85000	185000
7	Audi	A4	2010	185000	225000

La consulta en SQL para actualizar el kilometraje sería:

```
UPDATE Auto SET kilometraje = 195000 WHERE id = 7;
```

```
mysql> UPDATE Auto SET kilometraje = 195000 WHERE id = 7;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM Auto;
+----+-----+-----+-----+-----+-----+
| id | Marca    | subMarca | Modelo | kilometraje | precioMXP |
+----+-----+-----+-----+-----+-----+
| 1  | Honda    | Civic    | 2015   | 67000       | 256000    |
| 2  | Volkswagen | Jetta    | 2009   | 148500      | 110000    |
| 3  | Toyota    | Sienna   | 2011   | 140500      | 149900    |
| 4  | Honda    | Fit      | 2015   | 110000      | 140000    |
| 5  | Mazda    | CX-5     | 2017   | 65000       | 265000    |
| 6  | Suzuki   | Vitara   | 2016   | 85000       | 185000    |
| 7  | Audi     | A4       | 2010   | 195000      | 225000    |
+----+-----+-----+-----+-----+-----+
7 rows in set (0.00 sec)
```

Esta consulta fue hecha con base en el `id`, pero si quisiéramos hacerlo basándonos en las características del auto, tendríamos:

```
UPDATE Auto SET kilometraje = 200000 WHERE Marca="Audi" AND subMarca="A4";
```

```
mysql> UPDATE Auto SET kilometraje = 200000 WHERE Marca="Audi" AND subMarca="A4";
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM Auto;
+----+-----+-----+-----+-----+-----+
| id | Marca   | subMarca | Modelo | kilometraje | precioMXP |
+----+-----+-----+-----+-----+-----+
| 1  | Honda   | Civic    | 2015   | 67000       | 256000    |
| 2  | Volkswagen | Jetta    | 2009   | 148500      | 110000    |
| 3  | Toyota   | Sienna   | 2011   | 140500      | 149900    |
| 4  | Honda   | Fit      | 2015   | 110000      | 140000    |
| 5  | Mazda   | CX-5     | 2017   | 65000       | 265000    |
| 6  | Suzuki   | Vitara   | 2016   | 85000       | 185000    |
| 7  | Audi    | A4       | 2010   | 200000      | 225000    |
+----+-----+-----+-----+-----+-----+
7 rows in set (0.00 sec)
```

Ahora, hagámoslo con Pandas.

Nuestro *dataframe* contiene la siguiente información:

	Marca	subMarca	Modelo	kilometraje	precio MXP
0	Honda	Civic	2015	67000	256000
1	Volkswagen	Jetta	2009	148500	110000
2	Toyota	Sienna	2011	140500	149900
3	Honda	Fit	2015	110000	140000
4	Mazda	CX-5	2017	65000	265000
5	Susuki	Vitara	2016	85000	185000
6	Audi	A4	2010	185000	225000

Para actualizar el kilometraje del Audi A4 a 195,000 kilómetros, usamos la siguiente instrucción:

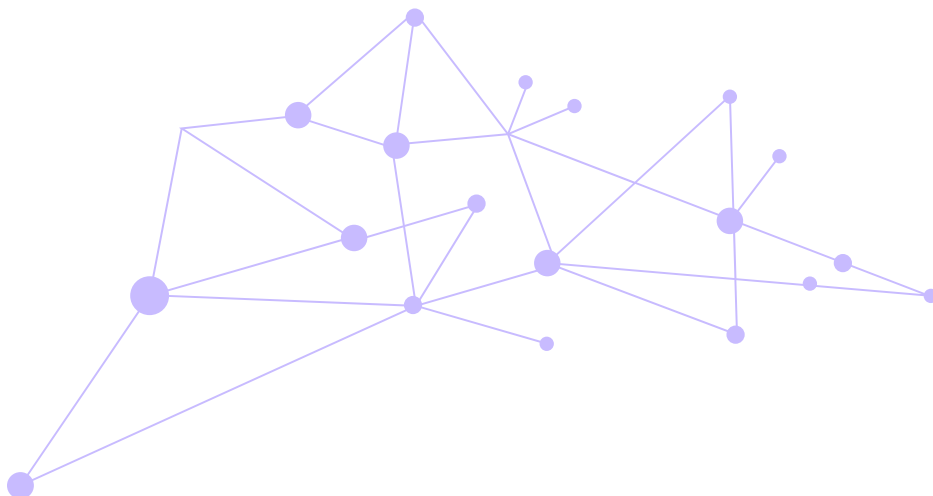
```
excel_autos.at[6,"kilometraje"] = 195000
```

Lo que arroja el siguiente resultado:

	Marca	subMarca	Modelo	kilometraje	precio MXP
0	Honda	Civic	2015	67000	256000
1	Volkswagen	Jetta	2009	148500	110000
2	Toyota	Sienna	2011	140500	149900
3	Honda	Fit	2015	110000	140000
4	Mazda	CX-5	2017	65000	265000
5	Susuki	Vitara	2016	85000	185000
6	Audi	A4	2010	195000	225000

Y para actualizar el kilometraje haciendo referencia a las características del auto, podemos hacerlo de la siguiente manera:

```
excel_autos.loc[(excel_autos["Marca"]== "Audi")  
& (excel_autos["subMarca"]=="A4"),"kilometraje"] = 200000
```



El resultado:

	Marca	subMarca	Modelo	kilometraje	precio MXP
0	Honda	Civic	2015	67000	256000
1	Volkswagen	Jetta	2009	148500	110000
2	Toyota	Sienna	2011	140500	149900
3	Honda	Fit	2015	110000	140000
4	Mazda	CX-5	2017	65000	265000
5	Susuki	Vitara	2016	85000	185000
6	Audi	A4	2010	200000	225000

■ Eliminar

La última de las operaciones básicas es la eliminación de datos. Para seguir con nuestra comparación, veamos primero cómo hacerlo en MySQL, y luego en Pandas. Recuerda que la operación de eliminar es muy sencilla, consiste en borrar permanentemente algunos registros.

Partimos de la siguiente información:

```
mysql> SELECT * FROM Auto;
+----+-----+-----+-----+-----+-----+
| id | Marca      | subMarca | Modelo | kilometraje | precioMXP |
+----+-----+-----+-----+-----+-----+
| 1  | Honda      | Civic    | 2015   | 67000        | 256000    |
| 2  | Volkswagen | Jetta     | 2009   | 148500       | 110000    |
| 3  | Toyota     | Sienna    | 2011   | 140500       | 149900    |
| 4  | Honda      | Fit       | 2015   | 110000       | 140000    |
| 5  | Mazda      | CX-5      | 2017   | 65000        | 265000    |
| 6  | Suzuki     | Vitara    | 2016   | 85000        | 185000    |
| 7  | Audi       | A4        | 2010   | 200000       | 225000    |
+----+-----+-----+-----+-----+-----+
7 rows in set (0.00 sec)
```

Podemos eliminar un registro por medio de su `id`. Por ejemplo, si queremos eliminar el vehículo Toyota Sienna, podemos usar la instrucción `DELETE FROM`, de este modo:

```
DELETE FROM Auto WHERE id = 3;
```

El resultado será:

```
mysql> SELECT * FROM Auto;
+----+-----+-----+-----+-----+-----+
| id | Marca      | subMarca | Modelo | kilometraje | precioMXP |
+----+-----+-----+-----+-----+-----+
| 1  | Honda      | Civic    | 2015   | 67000       | 256000    |
| 2  | Volkswagen | Jetta    | 2009   | 148500      | 110000     |
| 4  | Honda      | Fit      | 2015   | 110000      | 140000     |
| 5  | Mazda      | CX-5     | 2017   | 65000       | 265000     |
| 6  | Suzuki     | Vitara   | 2016   | 85000       | 185000     |
| 7  | Audi       | A4       | 2010   | 200000      | 225000     |
+----+-----+-----+-----+-----+-----+
6 rows in set (0.01 sec)
```

Ahora, supongamos que queremos eliminar los autos que cuesten más de \$200,000 pesos mexicanos. La instrucción sería:

```
DELETE FROM Auto WHERE precioMXP > 200000;
```

El resultado:

```
+----+-----+-----+-----+-----+-----+
| id | Marca      | subMarca | Modelo | kilometraje | precioMXP |
+----+-----+-----+-----+-----+-----+
| 2  | Volkswagen | Jetta    | 2009   | 148500      | 110000     |
| 4  | Honda      | Fit      | 2015   | 110000      | 140000     |
| 6  | Suzuki     | Vitara   | 2016   | 85000       | 185000     |
+----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Ahora, hagámoslo en Pandas.

Eliminemos primero a la Sienna usando su índice. Observa la siguiente información:

	Marca	subMarca	Modelo	kilometraje	precio MXP
0	Honda	Civic	2015	67000	256000
1	VolksWagen	Jetta	2009	148500	110000
2	Toyota	Sienna	2011	140500	149900
3	Honda	Fit	2015	110000	140000
4	Mazda	CX-5	2017	65000	265000
5	Susuki	Vitara	2016	85000	185000
6	Audi	A4	2010	200000	225000

Vemos que el vehículo Toyota Sienna tiene el índice 2, podemos ejecutar la instrucción:

```
excel_autos.drop(2).reset_index(drop=True)
```

Con esto, eliminamos la fila con índice 2, y reiniciamos los índices para que no quedaran huecos. Es decir, que los índices queden de forma secuencial, desde el 0 hasta el 5. Observa el resultado a continuación:

	Marca	subMarca	Modelo	kilometraje	precio MXP
0	Honda	Civic	2015	67000	256000
1	VolksWagen	Jetta	2009	148500	110000
2	Honda	Fit	2015	110000	140000
3	Mazda	CX-5	2017	65000	265000
4	Susuki	Vitara	2016	85000	185000
5	Audi	A4	2010	200000	225000

Ahora, supongamos que queremos eliminar los autos que cuesten más de \$200,000 pesos mexicanos. En lugar de eliminar los que cuestan más de dicha cantidad, nos quedaremos con aquellos cuyo precio sea igual o menor a \$200,000. Para esto, ejecutamos:

```
Excel_autos2 = Excel_autos.loc[Excel_autos["precio MXP"] <= 200000]
Excel_autos2
```

El resultado:

	Marca	subMarca	Modelo	kilometraje	precio MXP
1	Volkswagen	Jetta	2009	148500	110000
2	Toyota	Sienna	2011	140500	149900
3	Honda	Fit	2015	110000	140000
5	Susuki	Vitara	2016	85000	185000

Introducción a pandaSQL para consultas SQL sobre *dataframes*

Hasta este punto, ya hemos revisado cómo realizar operaciones básicas tanto en SQL como en Pandas. Ahora exploraremos una biblioteca muy interesante llamada **pandasql**, que permite realizar consultas SQL directamente sobre un *dataframe*, como si se tratara de una tabla en un sistema de bases de datos.

Veamos un par de ejemplos utilizando nuestro *dataframe* de autos que usamos en el subtema anterior.

```
excel_autos
```

	Marca	subMarca	Modelo	kilometraje	precio MXP
0	Honda	Civic	2015	67000	256000
1	VolksWagen	Jetta	2009	148500	110000
2	Toyota	Sienna	2011	140500	149900
3	Honda	Fit	2015	110000	140000
4	Mazda	CX-5	2017	65000	265000
5	Susuki	Vitara	2016	85000	185000
6	Audi	A4	2010	200000	225000

Para poder utilizar esta biblioteca, lo primero que debes hacer es instalarla. Utiliza el siguiente comando:

```
!pip install pandasql
```

Luego, debes importar `sqldf`, que es la función principal que utilizaremos. Esta función permite ejecutar consultas en sintaxis SQL, tratando los nombres de los *dataframes* disponibles como si fueran tablas dentro de un sistema gestor de bases de datos.

```
from pandasql import sqldf
```

La función `sqldf` regresará un *dataframe* que contiene el resultado de la consulta. Por ejemplo:

```
rdf = sqldf("SELECT * FROM excel_autos;")
rdf
```

	Marca	subMarca	Modelo	kilometraje	precio MXP
0	Honda	Civic	2015	67000	256000
1	Volkswagen	Jetta	2009	148500	110000
2	Toyota	Sienna	2011	140500	149900
3	Honda	Fit	2015	110000	140000
4	Mazda	CX-5	2017	65000	265000
5	Susuki	Vitara	2016	85000	185000
6	Audi	A4	2010	200000	225000

Como puedes ver, nos regresa exactamente el mismo *dataframe* que teníamos.

Ahora, veamos algunas de las consultas sobre los autos revisadas en el subtema anterior.

La consulta para mostrar los autos Honda sería la siguiente:

```
SELECT * FROM excel_autos WHERE Marca = "Honda";
```

```
query = 'SELECT * FROM excel_autos WHERE Marca = "Honda";'
rdf = sqldf(query)
rdf
```

	Marca	subMarca	Modelo	kilometraje	precio MXP	
0	Honda	Civic	2015	67000	256000	
1	Honda	Fit	2015	110000	140000	

Si quisiéramos solo algunos atributos del resultado, lo podemos hacer con la siguiente consulta:

```
SELECT subMarca, Modelo FROM excel_autos WHERE Marca = "Honda";
```

```
query = 'SELECT subMarca, Modelo FROM excel_autos WHERE Marca = "Honda";'
rdf = sqldf(query)
rdf
```

	subMarca	Modelo	
0	Civic	2015	
1	Fit	2015	

Si queremos solo algunos atributos de todos los autos, utilizamos:

```
SELECT subMarca, Modelo FROM excel_autos;
```

```
query = 'SELECT subMarca, Modelo FROM excel_autos;'
rdf = sqldf(query)
rdf
```

	subMarca	Modelo	
0	Civic	2015	
1	Jetta	2009	
2	Sienna	2011	
3	Fit	2015	
4	CX-5	2017	
5	Vitara	2016	
6	A4	2010	

Si quisiéramos, por ejemplo, los autos Honda con kilometraje menor a 100,000 kilómetros, usamos:

```
SELECT * FROM excel_autos WHERE marca = "Honda" and
kilometraje < 100000;
```

```
query = 'SELECT * FROM excel_autos WHERE marca = "Honda" and kilometraje < 100000;'  
rdf = sqldf(query)  
rdf
```

	Marca	subMarca	Modelo	kilometraje	precio MXP	
0	Honda	Civic	2015	67000	256000	

¿Cuándo es buena idea usar pandasql?

La biblioteca pandasql permite trabajar con datos usando el **Lenguaje de Consulta de Datos** (DQL, por sus siglas en inglés), un subconjunto de SQL. En otras palabras, con pandasql podemos ejecutar consultas sobre los datos almacenados en una base de datos para recuperar la información necesaria. En particular, podemos acceder, extraer, filtrar, ordenar, agrupar, unir, agregar los datos y realizar operaciones matemáticas o lógicas.

¿Cuándo no es buena idea usar pandasql?

Pandasql solo permite ejecutar consultas de tipo DQL, es decir, aquellas destinadas exclusivamente a la lectura de datos. Por lo tanto, **no** se puede utilizar para modificar tablas ni para realizar operaciones como actualizar, eliminar o insertar datos.

Si deseas profundizar en el uso de pandasql, en la sección de Material adicional encontrarás algunos recursos que te ayudarán a ampliar tus conocimientos sobre esta biblioteca.



Para terminar...

A lo largo de esta lección, hemos explorado cómo herramientas potentes como **SQL** y **Pandas** pueden trabajar de manera conjunta para facilitar el análisis de datos, tanto en entornos estructurados como en escenarios más flexibles y dinámicos. Comprender las analogías entre ambas no solo fortalecerá tus conocimientos técnicos, sino que también ampliará tus horizontes como profesional en el manejo de información.

Hoy en día, los datos no siempre se almacenan en bases de datos relacionales. En muchos casos (ya sea en empresas, negocios o proyectos personales) es común trabajar con archivos sueltos, datos alojados en la nube, hojas de cálculo o reportes automatizados. En estos contextos, Pandas se convierte en una herramienta muy poderosa gracias a su enfoque en el análisis de datos desde Python.

Lo más interesante es que ahora puedes combinar lo aprendido: con **pandasql**, puedes aplicar directamente tus conocimientos de SQL para hacer consultas sobre *dataframes* de Pandas, lo que permite un manejo de datos más flexible y eficiente.

Antes de continuar, reflexiona:

-  ¿Cómo podrías aplicar **pandasql**, en un proyecto real que ya tengas o estés planeando, para agilizar el análisis de datos?
-  ¿Qué ventaja te ofrece, como profesional, el poder trabajar tanto con bases de datos relacionales como con estructuras de datos en Python?

Continúa con el siguiente recurso, donde pondrás a prueba los conocimientos adquiridos en esta lección.

| Bibliografía adicional

● Documentación oficial de Pandas:

Pandas Development Team. (2025). *pandas.read_excel – pandas 2.2.3 documentation*. PyData. Recuperado el 07 de mayo de 2025 de: https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.read_excel.html

● Blogs:

Nilabh. (14 de octubre de 2024). *Pandasql - The Best Way to Run SQL Queries in Python* [Pandasql: La mejor manera de ejecutar consultas SQL en Python]. Analytics Vidhya. Recuperado el 07 de mayo de 2025 de: <https://www.analyticsvidhya.com/blog/2021/07/pandasql-best-way-to-run-sql-queries-and-codes-in-jupyter-notebook-using-python/>

GeeksforGeeks. (24 de abril de 2025). *Python | Pandas CRUD Operations* [Operaciones CRUD de Python | Pandas]. GeeksforGeeks. Recuperado el 07 de mayo de 2025 de: <https://www.geeksforgeeks.org/python-pandas-crud-operations/>

● Tutorial:

Kosourova, E. (15 de mayo de 2023). *How to Use SQL in pandas Using pandasql Queries* [Cómo usar SQL en pandas mediante consultas pandasql]. DataCamp. Recuperado el 07 de mayo de 2025 de: <https://www.datacamp.com/tutorial/how-to-use-sql-in-pandas-using-pandasql-queries>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 08 de mayo de 2025 y su disponibilidad depende del autor.

| Glosario

Dataframe: Estructura de datos bidimensional en Pandas, similar a una hoja de cálculo o una tabla en una base de datos, compuesta por filas e índices, y columnas con etiquetas.

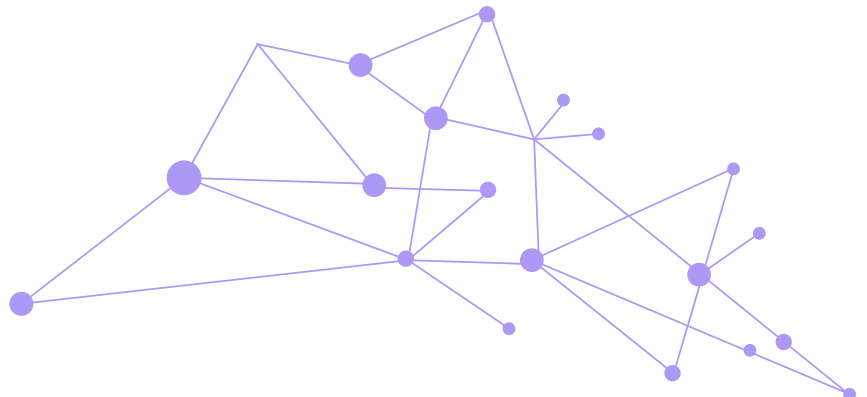
Data Manipulation Language (DML): Subconjunto del lenguaje SQL que incluye sentencias para insertar, actualizar, eliminar y recuperar datos en una base de datos.

Condición booleana: Expresión que devuelve True o False y se utiliza en Pandas para filtrar datos o realizar operaciones condicionales.

Pandas: Librería de Python especializada en el manejo y análisis de datos. Permite trabajar con estructuras como *dataframe* y series para procesar información de manera eficiente.

pandasql: Biblioteca de Python que permite ejecutar consultas SQL directamente sobre objetos *dataframe* de Pandas, utilizando una sintaxis familiar para quienes provienen del mundo de bases de datos relacionales.

Query: Instrucción escrita en SQL para solicitar información específica de una base de datos. En Pandas o pandasql, las *queries* pueden recuperar datos de un *dataframe* según condiciones o filtros.



Referencias bibliográficas

Elmasri, R. & Navathe, S. B. (2016). *Fundamentals of database systems* [Fundamentos de los sistemas de bases de datos] (7th ed.). Pearson Education.

Lamp, G. (20 de abril de 2016). *pandasql* (versión 0.7.3) [Paquete de Python]. PyPI. Recuperado el 07 de mayo de 2025 de: <https://pypi.org/project/pandasql/>

MySQL. (2024). *INSERT Statement* [Declaración INSERT]. MySQL 8.4 Reference Manual. Oracle. Recuperado el 07 de mayo de 2025 de: <https://dev.mysql.com/doc/refman/9.0/en/insert.html>

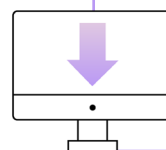
MySQL. (2024). *SELECT Statement* [Declaración SELECT]. MySQL 9.0 Reference Manual. Oracle. Recuperado el 07 de mayo de 2025 de: <https://dev.mysql.com/doc/refman/9.0/en/select.html>

MySQL. (2024). *UPDATE Statement* [Declaración UPDATE]. MySQL 9.0 Reference Manual. Oracle. Recuperado el 07 de mayo de 2025 de: <https://dev.mysql.com/doc/refman/9.0/en/update.html>

MySQL. (2024). *DELETE Statement* [Declaración DELETE]. MySQL 9.1 Reference Manual. Oracle. Recuperado el 07 de mayo de 2025 de: <https://dev.mysql.com/doc/refman/9.0/en/delete.html>

Pandas Development Team. (2025). *pandas.DataFrame*. PyData. Recuperado el 07 de mayo de 2025 de: <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 08 de mayo de 2025 y su disponibilidad depende del autor.



Descarga este documento en la sección “**Archivos adjuntos**” de la plataforma.